

TimeArch Brownfield Discovery

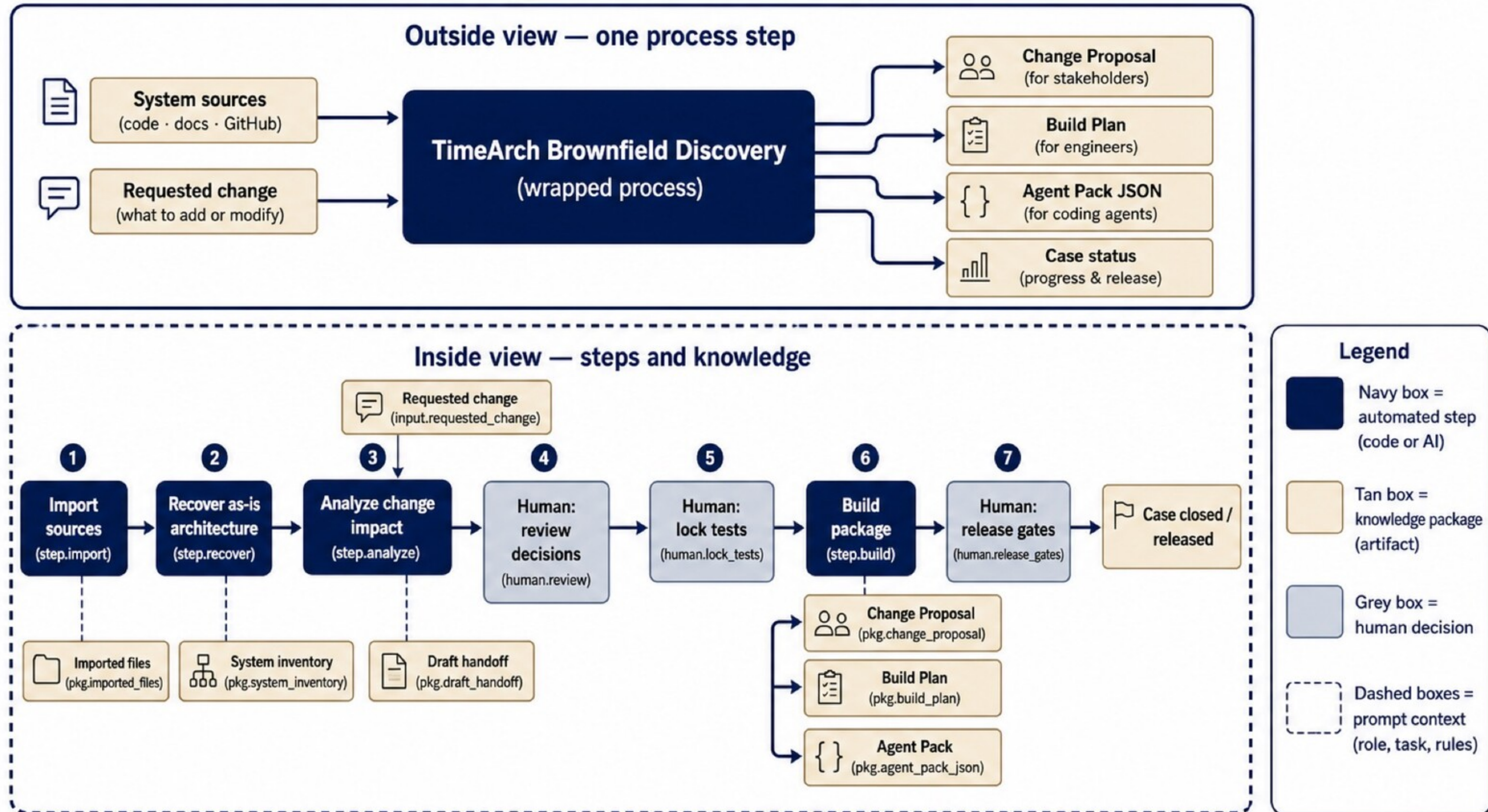
Process figures (KERKIS modeling)

Import Recover Change Released Closed

ResearchAnonymized/TimeArch · docs/modeling

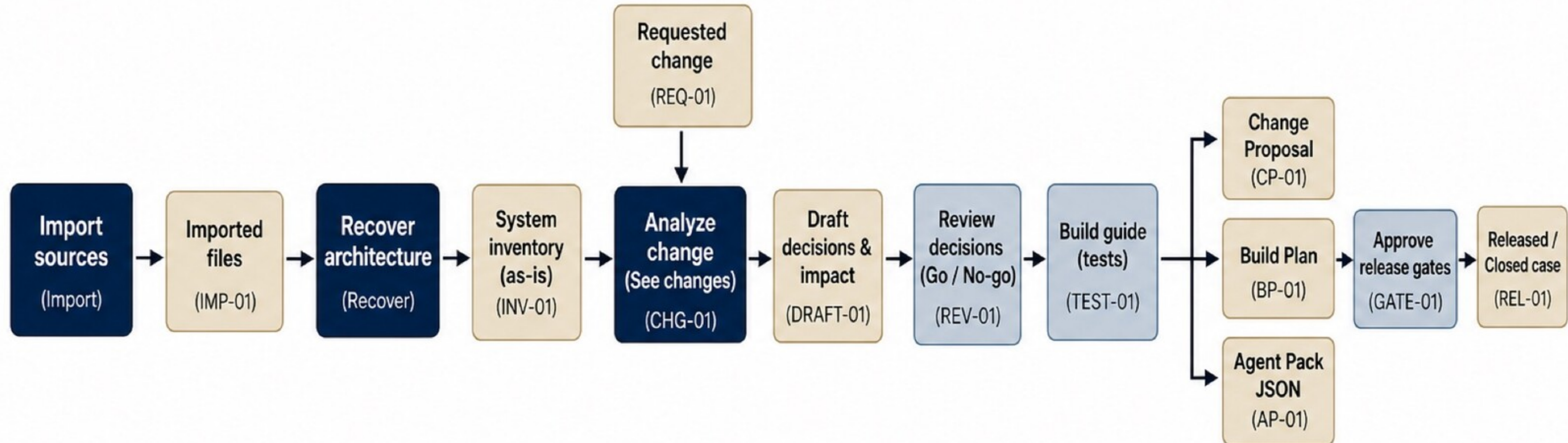
TimeArch Brownfield Discovery — one wrapped process

What neighbors see vs what happens inside



Brownfield path — step by step

How knowledge moves through TimeArch



Each step consumes the previous step's knowledge package. Humans approve before coding agents may implement.

Eligibility

You can start when you have the sources and a requested change to evaluate.

Ordering

Follow the sequence. Each step uses the output from the previous step.

Done when

The case is released or closed and all knowledge packages are stored in TimeArch.

Who does the work in TimeArch

Three kinds of actors — code, AI, and humans



Function (deterministic code)

Same input always gives same output. Used for import, recovery parsers, and building export files.

Examples

- Import from GitHub / upload
- Reverse-engineer as-is system
- Build Agent Pack JSON



Agent (AI / LLM)

Non-deterministic. Behavior comes only from injected role, task, and knowledge packages — not hard-coded persona.

Examples

- Map change to architecture
- Analyze blast radius
- Critic / quality check



Human (via the UI)

People make irreversible decisions.

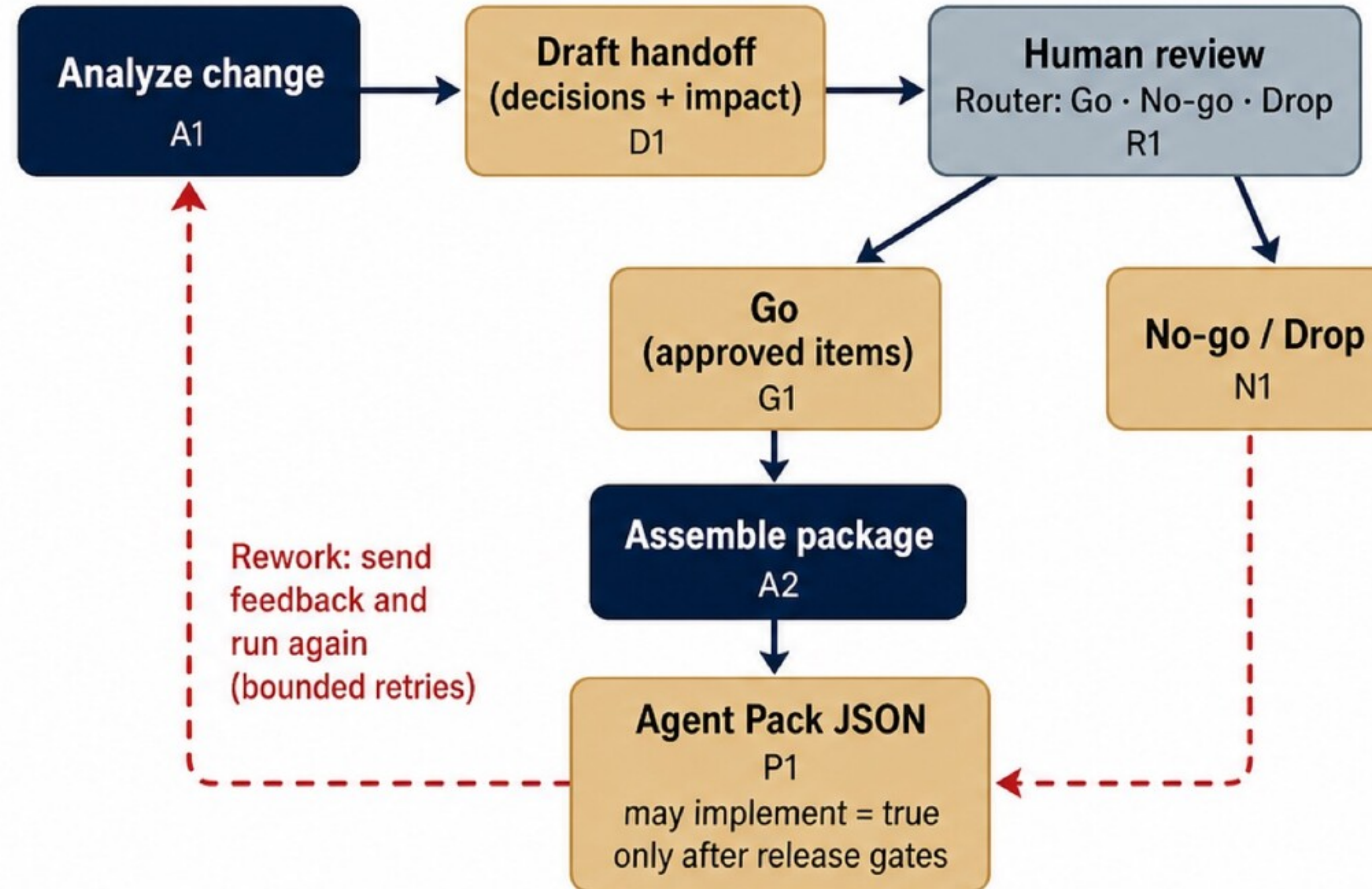
Examples

- Architect: Go / No-go on decisions
- Engineer: accept tests
- Approver: release gates

- 1 Substitutable** — swap code, AI, or human inside a step; neighbors only see the knowledge packages in and out.
- 2 Knowledge is not inside the actor** — roles and tasks are separate packages.

Human review and rework loop

How TimeArch routes Go vs No-go without a hard-coded flowchart



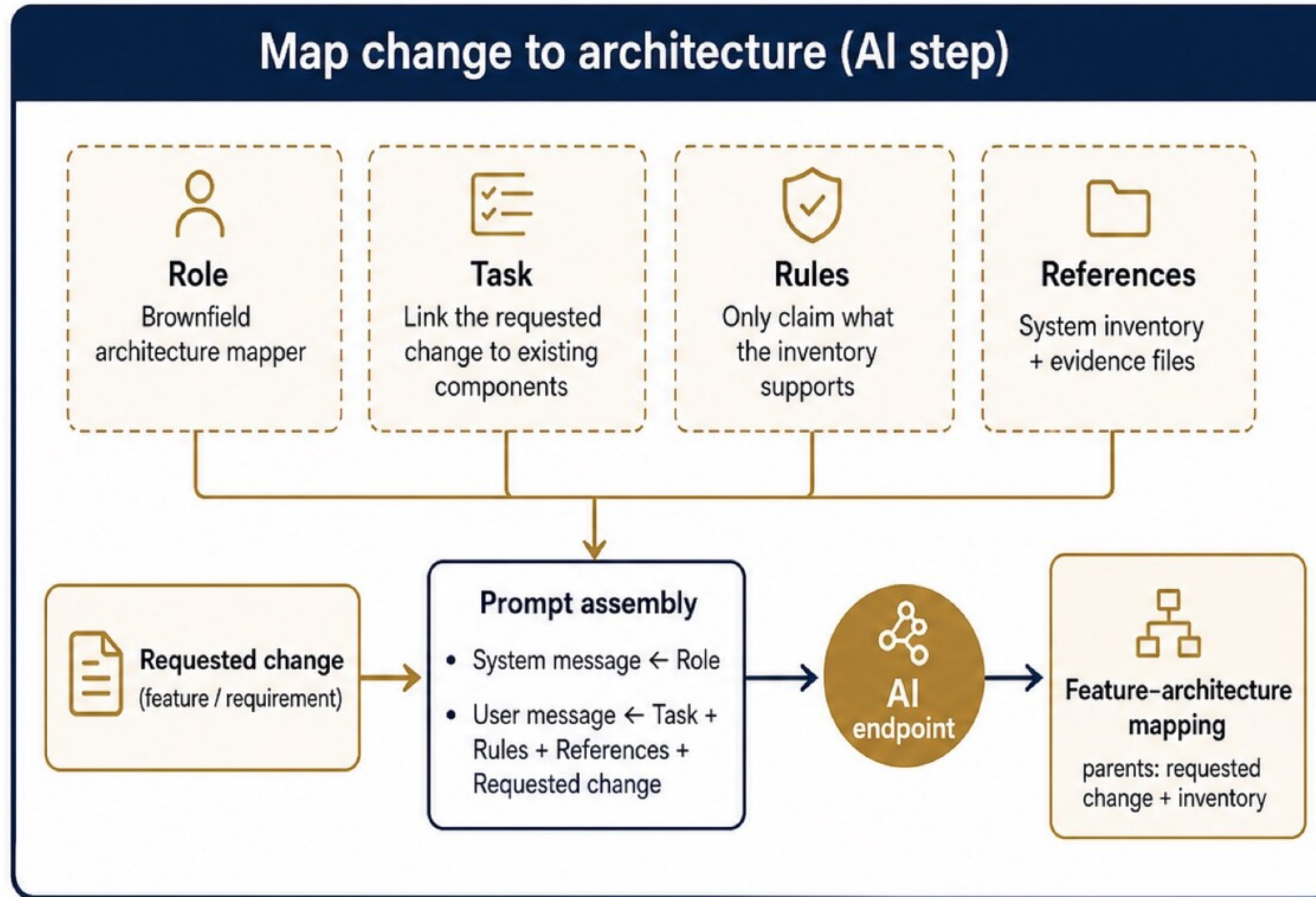
- **Routing:**
only Go items enter the Build Plan and Agent Pack
- **Rework:**
No-go sends the team back to analyze with feedback
- **Bounded:**
retries stop at an iteration limit
- **Safety:**
coding agents must stop if may_implement is false



Feedback is a typed knowledge package, not a chat dump.

One AI step in detail – Map change to architecture

How TimeArch builds the prompt from knowledge packages



Declares what it needs in and what it produces out



Starts only when required inputs exist



Records lineage (parents) on every new package

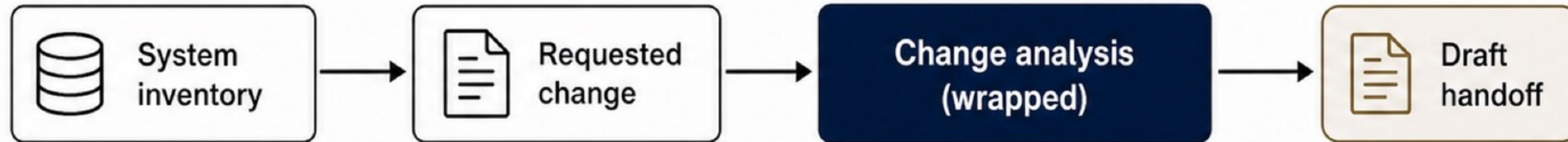


Can be re-run with feedback up to a retry limit

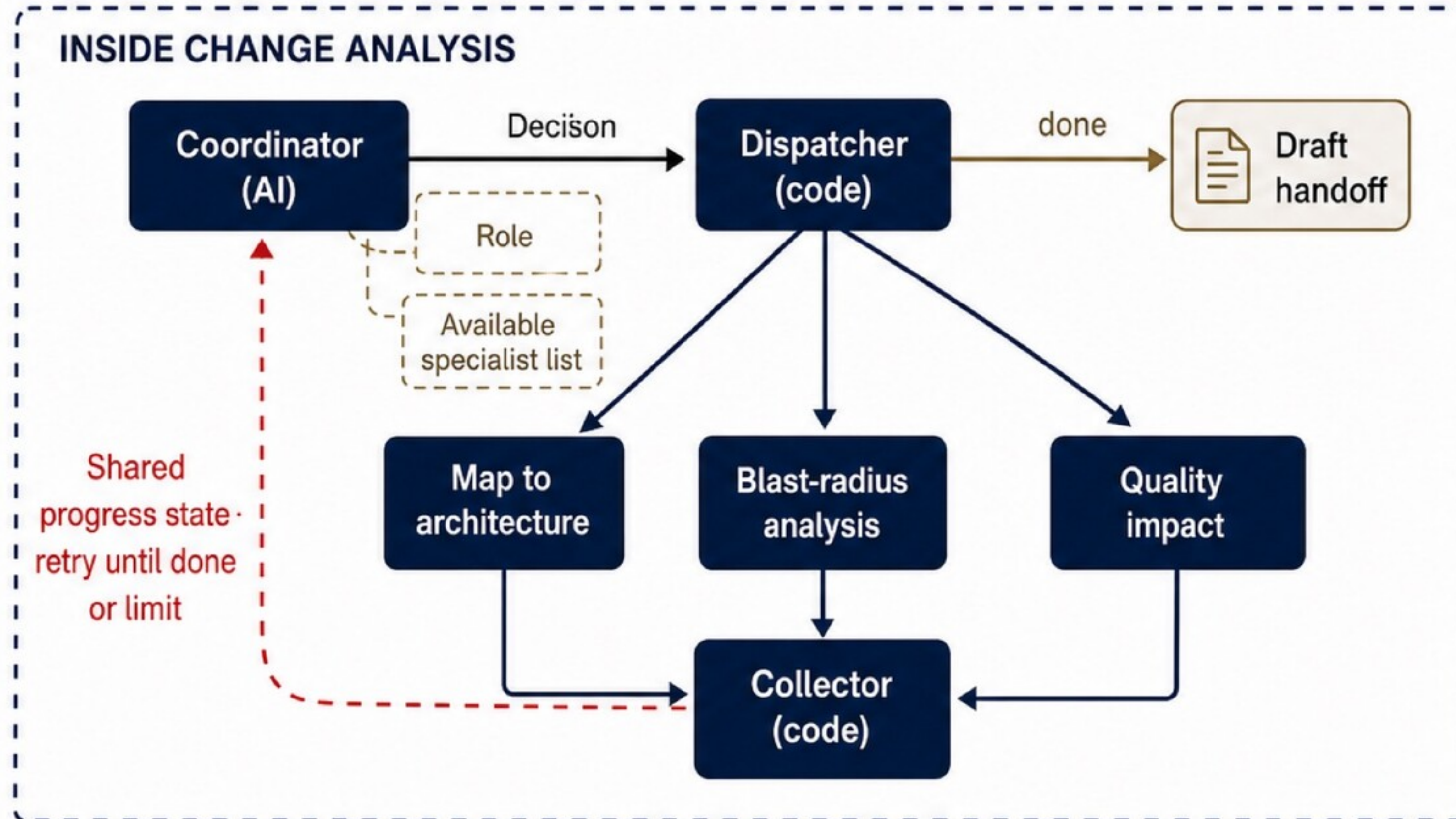
Nested analysis inside TimeArch

Change analysis as one step from outside; team of specialists inside

OUTSIDE VIEW



INSIDE CHANGE ANALYSIS



- Coordinator chooses what to run next
- Specialists are not hard-coded — they come from a registry package
- Loop is bounded by a retry limit
- Outside callers only see *Draft handoff* in/out